CS 3244

Machine Learning

Week 10 Pre-Class
Decision Trees

10 pre

Kan Min-Yen

Background Photo credits: Rafiq Mirza, Luan Ahn, Rosmarie Voegtli @ Flickr

Recap and Forecast

- Understand why deep learning was enabled by certain key technological milestones;
- Understand the Recurrent and Convolutional NN architectures (shortened as RNNs and CNNs);
- Understand how DNNs regularize and optimize via dropout, pretraining and choice of activation function;
- Relate embeddings as an a method to incorporate prior knowledge as a hidden layer;
- Obtain practical experience in constructing DNNs with PyTorch.

Learning Outcomes for this week

- Understand and implement decision trees;
- Master the concept of Information Gain as a means of choosing optimal attributes for decisions;
- Investigate methods for Ensembling as a means of integrating multiple classifiers;
- Examine Boosting and Bagging as basic techniques for extending ensembles;
    - Understand AdaBoost and Random Forests as extensions of Decision Trees using both techniques.

Decision Trees

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

How do people make decisions?

Well, let's examine some recent choices you made:

Deciding what to eat in the canteen.

What module to study for?

Would we say it's similar to how linear models make a decision (i.e., weighing each factor in a linear sum?)

No. People often focus on just a few dimensions of the evidence.

Decision trees embody this paradigm

Programming

Imperative programming control flow also embodies this

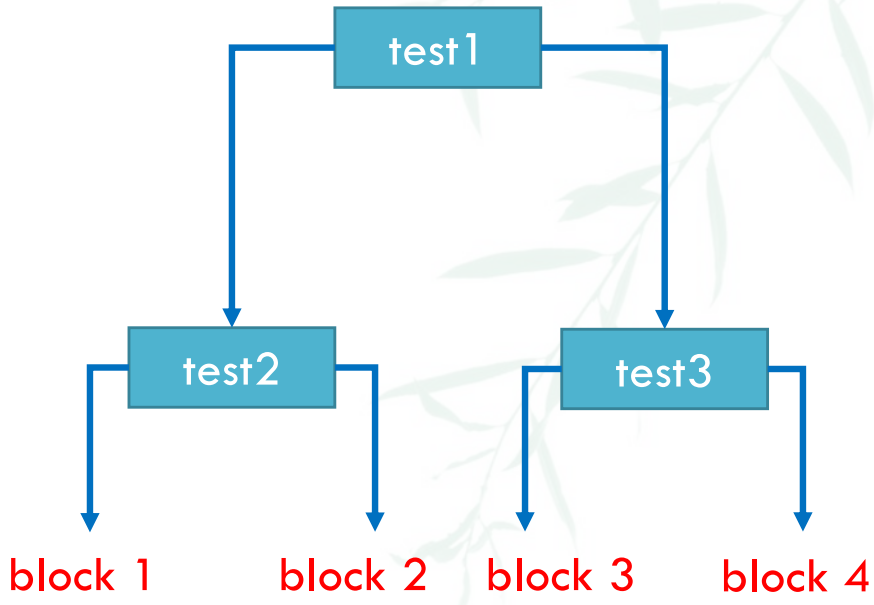
```
if test1:
  if test2:
    # block 1
  else:
    # block 2
else:
  if test3:
    # block 3
  else:
    # block 4
```

Programming

Imperative programming control flow also embodies this

```
if test1:
   if test2:
      # block 1
   else:
      # block 2
else:
   if test3:
      # block 3
   else:
      # block 4
```

This is a decision tree!



Parts of a tree

1. Internal nodes are tests

   In most systems, a node tests exactly one component of x, e.g., x_1

2. A branch in the tree corresponds to a test result

   A branch corresponds to an attribute value or a range of attribute values

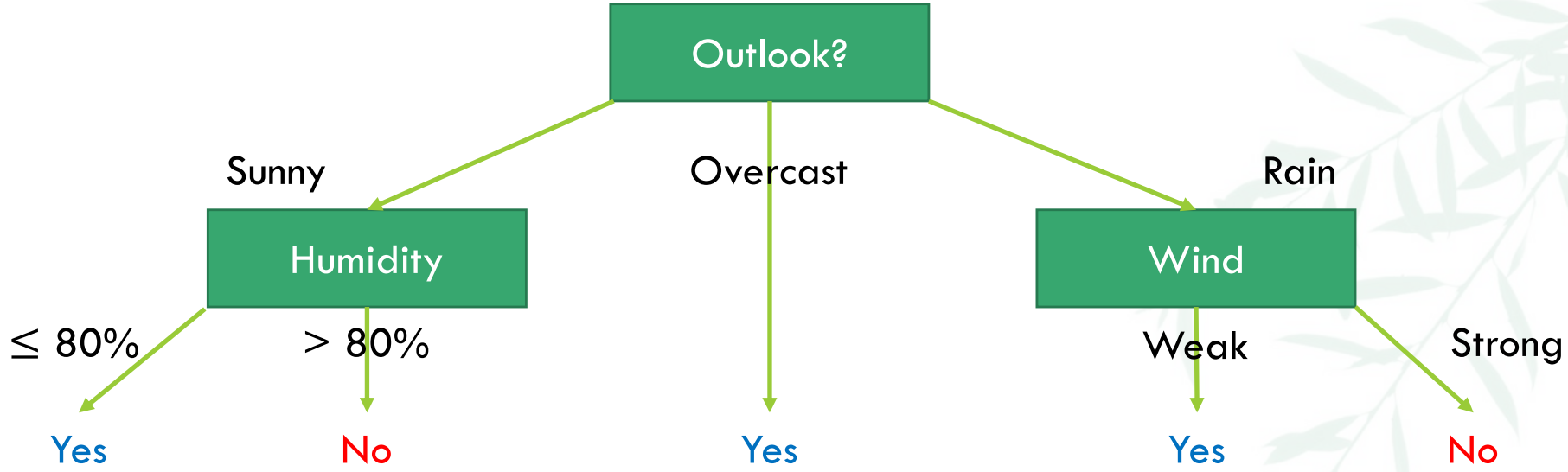
3. Each leaf node assigns
   - A class $y \in \{enum\}$: Decision tree
   - Real value $y \in \mathbb{R}$: Regression tree

Photo credits: Wikipedia

Shall we play tennis?

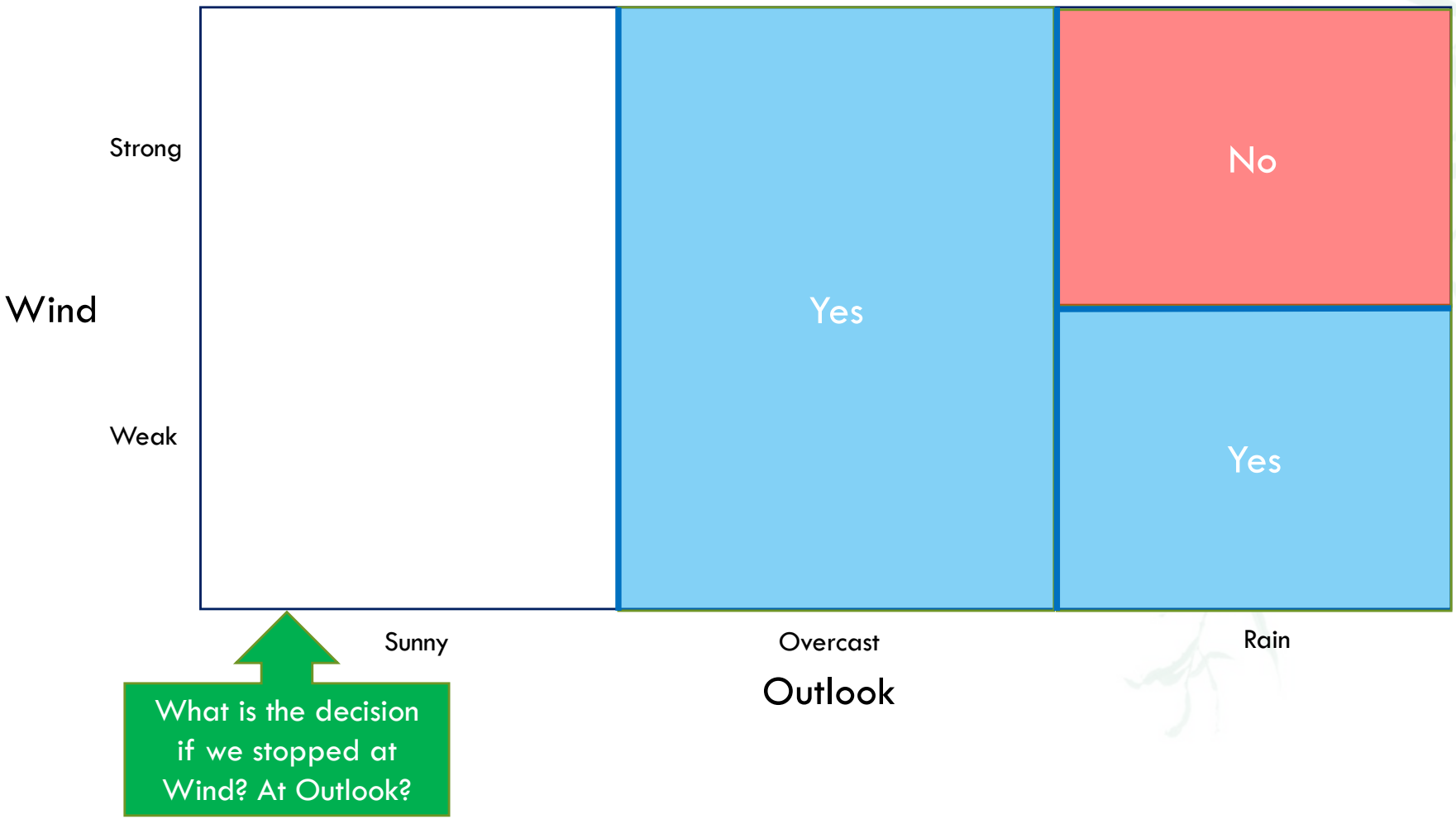
Day	Outlook	Temperature	Humidity	Wind	Play Tennis?
1	Sunny	Hot	95%	Weak	No
2	Sunny	Hot	92%	Strong	No
3	Overcast	Hot	95%	Weak	Yes
4	Rain	Mild	90%	Weak	Yes
5	Rain	Cool	60%	Weak	Yes
6	Rain	Cool	65%	Strong	No
7	Overcast	Cool	70%	Strong	Yes
8	Sunny	Cool	70%	Weak	Yes

Shall we play tennis?



Day	Outlook	Temperature	Humidity	Wind	Play Tennis?
x	Sunny	Hot	72%	Weak	?

Decision boundary



Axis perpendicular (recursive) decision boundaries

Analyzing Decision Trees

How expressive is the $\mathcal{H}$ of decision trees?

They can express any row $(x_1, x_2, \ldots, x_n, y)$ in the data set, trivially (how?).

You can always have $J_{train} = 0$ for a decision tree, if there is no noise in y.

What's the problem with a learned decision tree, in terms of learning theory?

Overfitting, high variance. How can we counter this?

3 Important Questions

<u>Answer</u>: Make smaller trees – lower the variance

As a result, prefer important decisions at the top

1. How do we pick a feature to split on?
2. How do we discretize continuous features?

Prune back complete trees*

3. How do we decide where to stop pruning?

* - we could decide when to stop growing the tree, but in practice this doesn't work well.

Decision tree learning

Aim: find a small tree minimizing J_{train}

Idea: (recursively) choose most significant attribute as root of (sub) tree

```
function DTL(examples, attributes, default) returns a decision tree
    if examples is empty then return default
    else if all examples have the same classification then return the classification
    else if attributes is empty then return MODE(examples)
    else
        best ← CHOOSE-ATTRIBUTE(attributes, examples)
        tree ← a new decision tree with root test best
        for each value v_i of best do
            examples_i ← {elements of examples with best = v_i}
            subtree ← DTL(examples_i, attributes − best, MODE(examples))
            add a branch to tree with label v_i and subtree subtree
        return tree
```


Algorithm credits: AIMA, Stuart and Russell

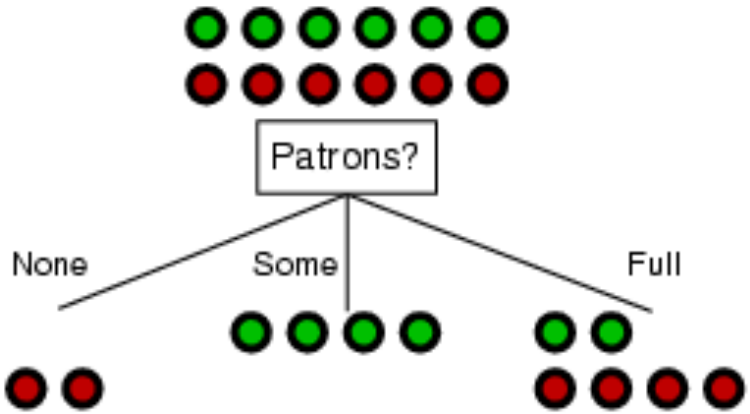
DTL: Picking Features via Entropy

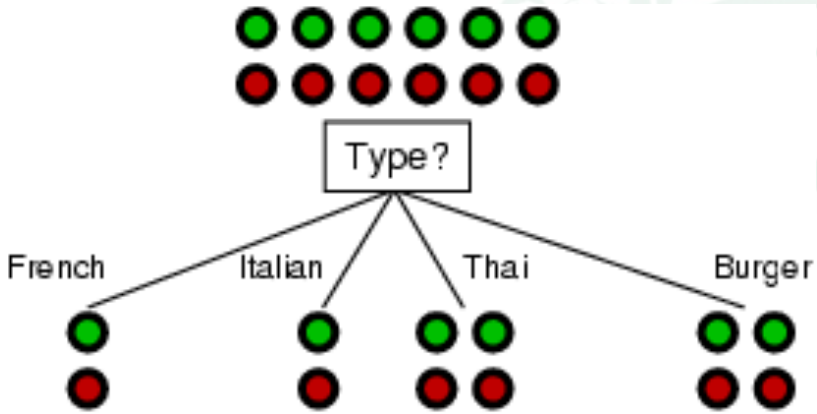
CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Choosing an attribute

Idea: a good attribute splits the training set into subsets that are (ideally) "all positive" or "all negative"





Patrons? is a better choice. Why?

Purity of the subproblems.

1. How do we pick a feature to split on?

To implement `Choose_Attribute` in DTL for enumerated feature sets with C possible values, we first need a concept of purity. We'll use entropy:

$$H(X) = -\sum_{i=0}^{C} p \log_2(p)$$

Example: for a training set containing p positive examples and n negative examples:

$$H\left(\frac{p}{p+n}, \frac{n}{p+n}\right) = -\frac{p}{p+n}\log_2\left(\frac{\mathrm{p}}{\mathrm{p}+\mathrm{n}}\right) - \frac{n}{p+n}\log_2\left(\frac{\mathrm{n}}{\mathrm{p}+\mathrm{n}}\right)$$

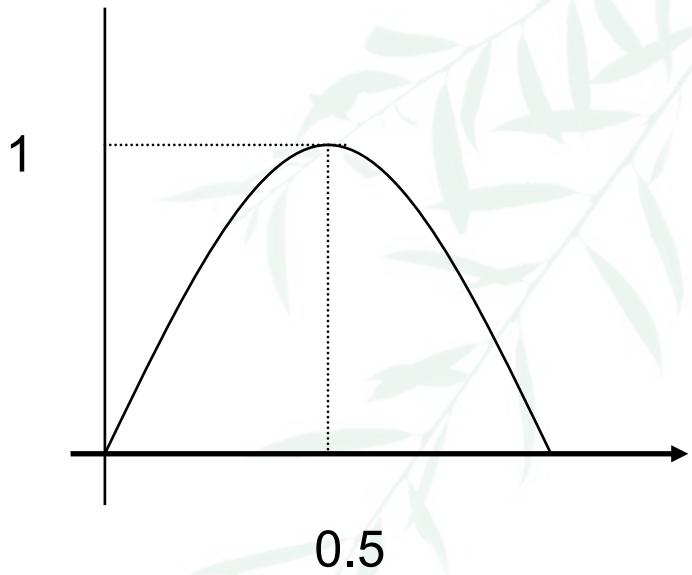
Entropy curve

For $\frac{p}{p+n}$, the 2-class entropy is:

0 when $\frac{p}{p+n}$ is 0

1 when $\frac{p}{p+n}$ is 0.5

0 when $\frac{p}{p+n}$ is 1



- monotonically increasing between 0 and 0.5
- monotonically decreasing between 0.5 and 1

Information gain

A chosen feature x_i divides the example set S into subsets $S_1, S_2, \ldots, S_c$ according to the C_i distinct values for x_i.

The entropy then reduces to the entropy of the subsets $S_1, S_2, \ldots, S_c$:

$$\text{remainder}(S, x_i) = \sum_{j=1}^{C_i} \frac{|S_j|}{|S|} H(S_j)$$

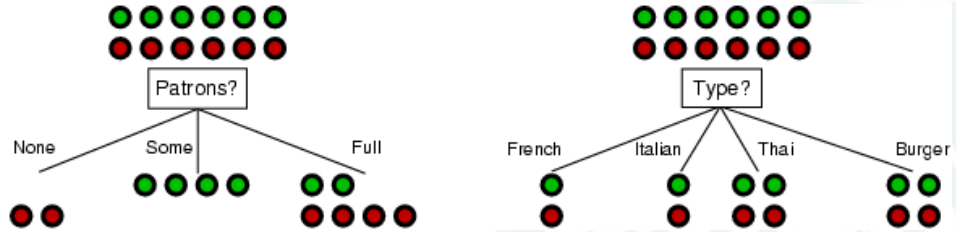
Information Gain (IG; "reduction in entropy") from knowing the value of x_i. Choose the attribute with the largest IG:

$$\text{IG}(S, x_i) = H(S) - \text{remainder}(S, x_i)$$

Information gain

For the training set at the root,

$$p = n = 6, H\left(\frac{6}{12}, \frac{6}{12}\right) = 1 \text{ bit.}$$



Consider the attributes *Patrons* and *Type*:

$$IG(Patrons) = 1 - \left[\frac{2}{12}H(0,1) + \frac{4}{12}H(1,0) + \frac{6}{12}H\left(\frac{2}{6}, \frac{4}{6}\right)\right] = 0.538 \text{ bits}$$

$$IG(Type) = 1 - \left[\frac{2}{12}H\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{2}{12}H\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{4}{12}H\left(\frac{2}{4}, \frac{2}{4}\right) + \frac{4}{12}H\left(\frac{2}{4}, \frac{2}{4}\right)\right] = 0 \text{ bits}$$

Patrons has the highest IG, and so is chosen by DTL as the root.

DTL: Discretizing and Pruning

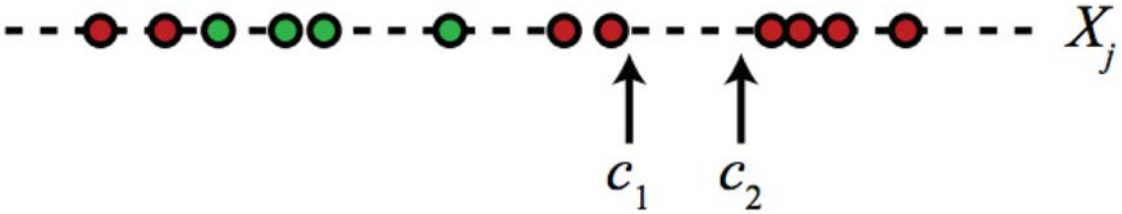
CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

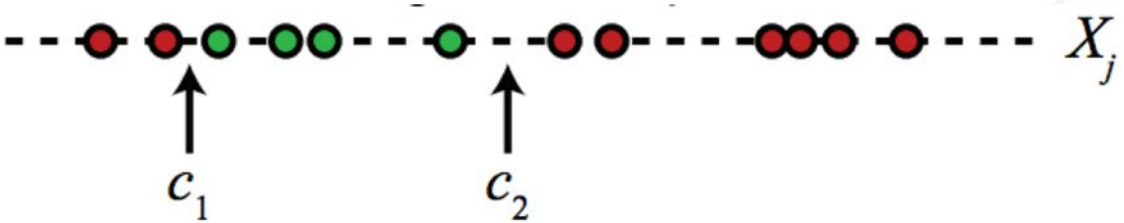
2. How do we discretize continuous features?

How can we choose among an infinite number of split points for a continuous feature x_j?

1. Moving split points between two observed values has no effect on information gain; use the midpoint.



2. Only splitting between examples from different labels y can be optimal for information gain.



3. How do we decide where to stop pruning?

Answer: Use validation!

Remove node and replace by MODE of labels y, if pruned tree performs better than original over validation set.

Photo credits: Vivian Garrido @ Wikipedia

Summary: Analyzing Decision Tree Learning

The good

Interpretable: Intuitive for data exploration.

Efficient: both in training (greedy search) and testing

Discards irrelevant features: through use of Information Gain.

The bad

Instability: high variance, susceptible to small fluctuations.

Hard decision boundaries: by default no probabilistic interpretation of boundaries

Axis perpendicular decisions: doesn't capture interaction between features.